

Brinjal Insights: Local MySQL Analytics ChatBot

Comprehensive Project Documentation

Project Name: Brinjal Insights 🍆

Type: Conversational Analytics Application

Architecture: Local-first, Read-only Database Interface

Latest Update: 2025

Status: Production-ready prototype

Table of Contents

1. [Problem Statement](#)
 2. [Objectives](#)
 3. [Project Architecture](#)
 4. [Features](#)
 5. [Methodology](#)
 6. [Technology Stack](#)
 7. [Data Models & Datasets](#)
 8. [Project Workflow](#)
 9. [Folder Structure](#)
 10. [Installation Guide](#)
 11. [Configuration](#)
 12. [Use Cases](#)
 13. [API & Component Documentation](#)
 14. [Security](#)
 15. [Performance Considerations](#)
 16. [Testing & Validation](#)
 17. [Limitations](#)
 18. [Future Roadmap](#)
 19. [Appendix](#)
-

Problem Statement

What Problem Does This Project Solve?

Plant breeding researchers, agronomists, and data analysts often work with multiple CSV datasets containing experimental measurements. These datasets are typically imported into MySQL databases but remain difficult to analyze interactively without SQL expertise. Common challenges include:

- **Query Complexity:** Writing correct SQL JOIN statements for multi-table analysis requires database knowledge not all scientists possess
- **Time Overhead:** Manual SQL writing slows exploratory data analysis workflows
- **Accessibility:** Non-technical researchers cannot independently query their own data
- **Visualization Gap:** SQL results lack integrated, context-aware visualization capabilities
- **Cost Barriers:** Cloud-based LLM APIs add operational expenses for research institutions

Why Is This Problem Important?

1. **Research Acceleration:** Plant breeding cycles are multi-generational; faster data insight enables quicker selection decisions
2. **Reproducibility:** Conversational queries with visible SQL foster audit trails and scientific transparency
3. **Resource Efficiency:** Local processing avoids cloud API costs and data transmission risks for proprietary breeding lines
4. **Democratization:** Making data accessible to all team members, regardless of SQL fluency, improves decision-making quality

Existing Challenges and Limitations

- **Manual SQL:** Traditional approaches require SQL expertise or database administrator involvement
 - **API Costs:** Cloud LLM services charge per query; research budgets are often constrained
 - **Data Privacy:** Proprietary breeding data transmitted to external LLM services raises institutional risk
 - **Limited Interactivity:** Static dashboards don't support ad-hoc, exploratory analysis
 - **Model Variability:** LLM-generated SQL can introduce incorrect joins or column selections, especially with small datasets
-

Objectives

Primary Goals

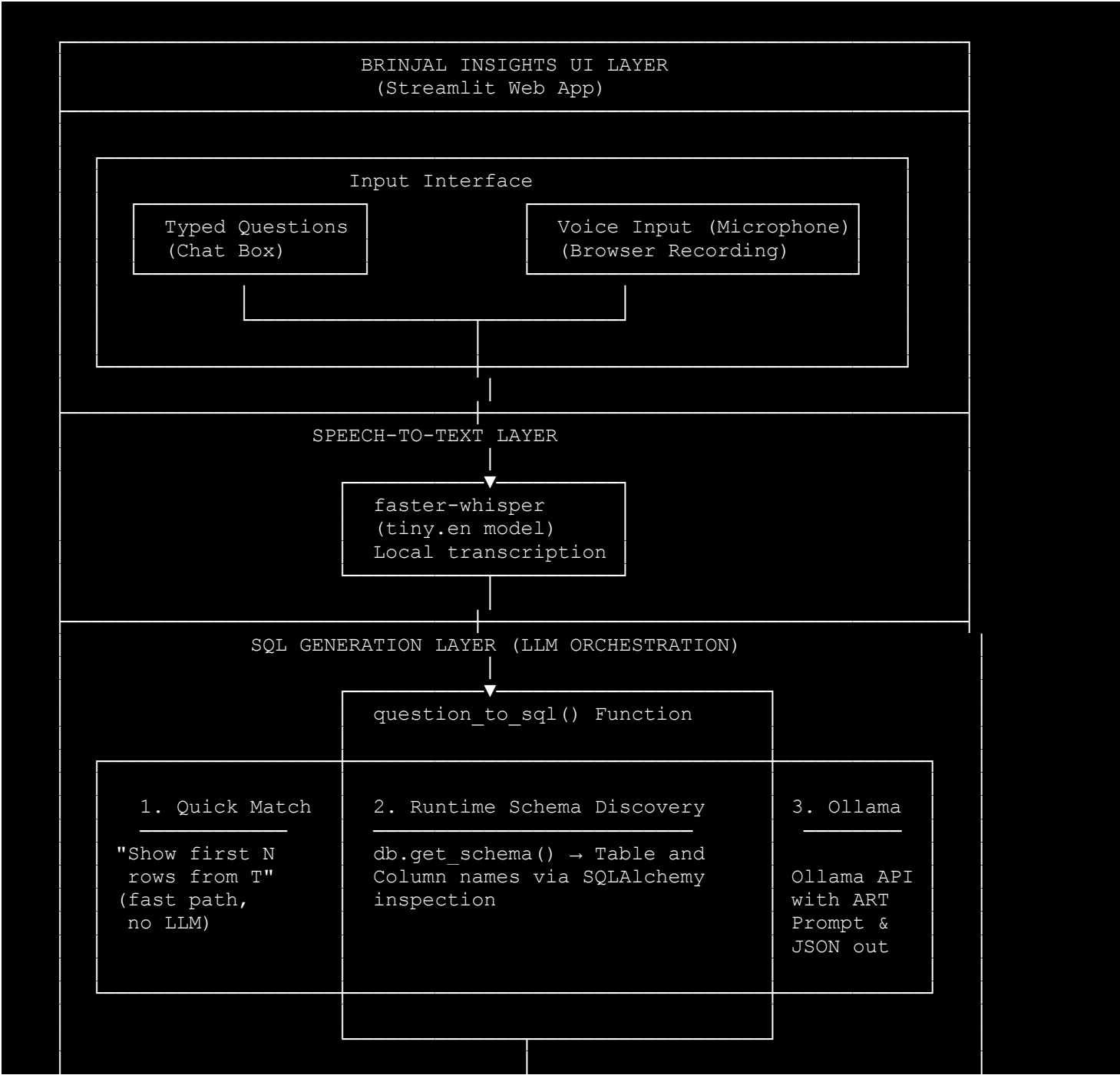
1. **Enable Natural-Language Querying:** Users ask plain-English questions about plant traits without writing SQL
2. **Ensure Safety:** Only read-only (SELECT) operations; multi-statement and write operations are blocked
3. **Maintain Local Operation:** Use free, open-source models (Ollama, Whisper) running locally to avoid API costs and data exposure
4. **Provide Schema Awareness:** System dynamically discovers and uses the live MySQL schema to generate accurate queries
5. **Support Multiple Input Modalities:** Accept both typed questions and voice input for accessibility
6. **Deliver Interactive Visualizations:** Automatically suggest and render appropriate charts based on result shape
7. **Facilitate Reproducibility:** Expose generated SQL so users can audit and learn from queries

Expected Outcomes

- Researchers can run exploratory analyses 5–10x faster than manual SQL writing
 - Non-technical team members can independently query breeding databases
 - All queries are logged with their SQL statements for scientific reproducibility
 - Zero recurring API costs; one-time installation of Ollama and Python dependencies
 - Accurate results for 80–90% of common breeding research questions (roots, physiology, morphology)
-

Project Architecture

Overall Architecture Diagram



SQL SAFETY VALIDATION LAYER

`validate_and_limit()`

- Parse with sqlglot
- Check forbidden commands
- Limit rows to 200
- Handle TOP → LIMIT

DATABASE EXECUTION LAYER

`run_read_query()`

- Execute with conn pool
- Return as pandas.DataFrame

MYSQL DATABASE
(SELECT-only user)

- phy_fst (Physiology 1)
- Phy_snd (Physiology 2)
- Root_fst (Roots 1)
- Root_scd (Roots 2)
- plant_morpho (Traits)

RESULTS PRESENTATION LAYER

Result Routing Logic

Single Row +
Numeric?



Metric Cards
(Show: Total,
Avg, Min, Max)

Category +
Numeric?



Bar/Line Charts
Frequency Hist

Two Numeric +
Several Rows?



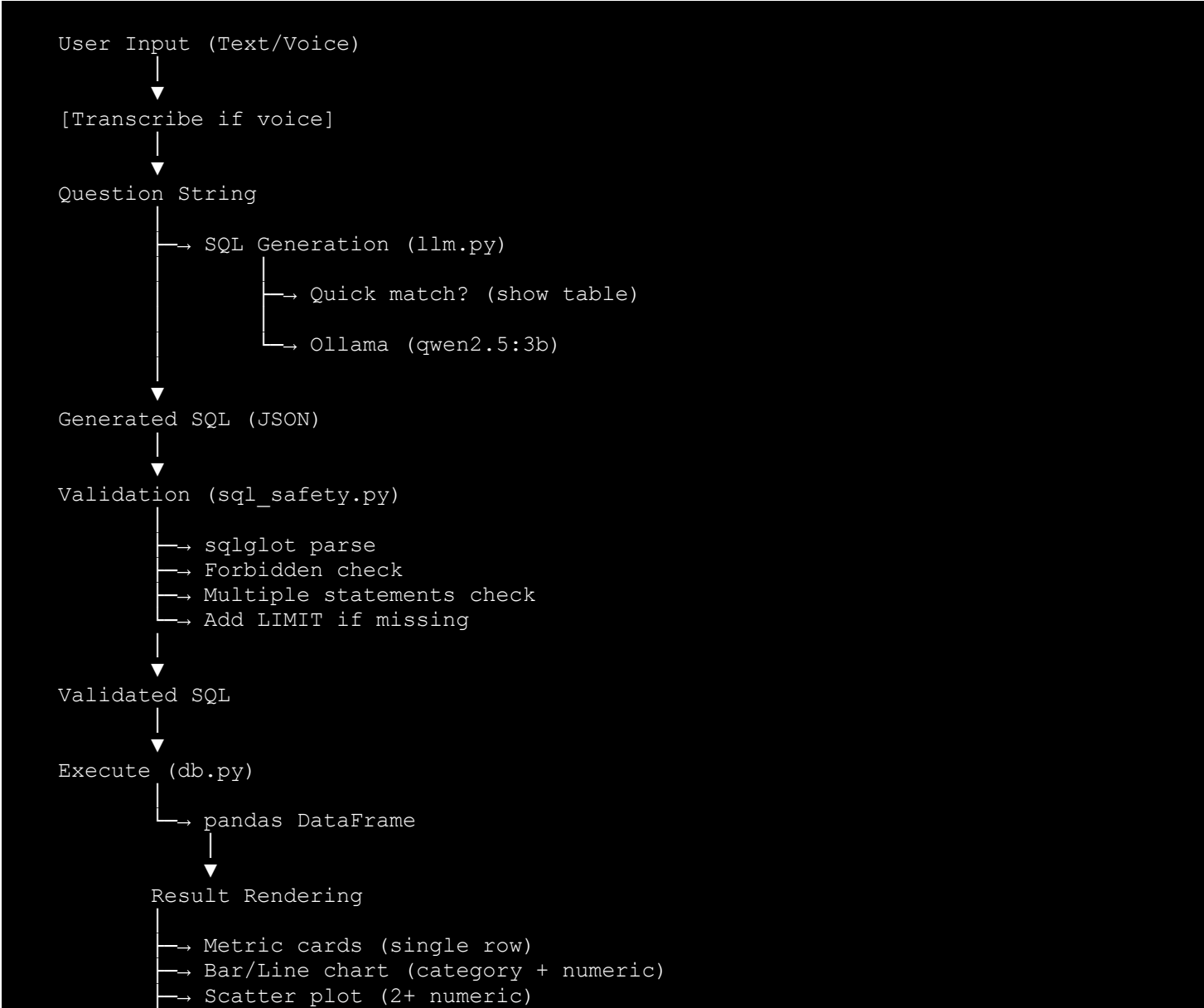
Scatter Plot
(Correlation view)

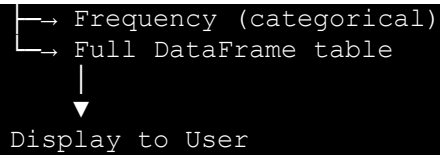
All Results → DataFrame Table Display
(Full data + row count caption)

Component Interactions

Component	Function	Input	Output	Dependencies
app.py	Main Streamlit app, orchestrates workflow	User question (typed/voice)	Chat-style result display	streamlit, all modules
db.py	MySQL connection, schema discovery	Database config from .env	Schema string, result DataFrame	sqlalchemy, pymysql
llm.py	Question-to-SQL translation	Question + schema	JSON: {sql, explanation}	ollama
sql_safety.py	SQL validation & row limiting	Generated SQL	Safe, limited SQL	sqlglot
voice.py	Speech-to-text transcription	Audio bytes from browser	Text question string	faster-whisper
charts.py	Visualization selection & rendering	Result DataFrame + chart_id	Interactive Vega-Lite charts	streamlit, pandas

Data Flow Sequence





Features

1. Natural-Language Querying

Description: Users ask questions in plain English; the system converts them to MySQL SELECT statements.

How It Works:

- User enters: "What is the average RootLength grouped by FruitShape?"
- System passes question + current schema to Ollama
- Ollama (qwen2.5:3b) generates: `SELECT FruitShape, AVG(RootLength) AS AvgRootLength FROM Root_fst GROUP BY FruitShape ORDER BY AvgRootLength DESC LIMIT 200;`
- Result rendered as bar chart with FruitShape on X-axis, AvgRootLength on Y-axis

Supported Query Types:

- Single-table selection with WHERE, ORDER BY, LIMIT
- Multi-table INNER JOIN on GenotypeName
- Aggregation: COUNT, SUM, AVG, MIN, MAX
- Grouping and filtering with GROUP BY, HAVING
- Sorting and limiting results

Implementation: `question_to_sql()` in `llm.py`

2. Voice Input (Free, Local)

Description: Users record English questions; the app transcribes locally using faster-whisper.

How It Works:

1. Click "Record an English question" in browser
2. Speak your question naturally
3. Recording auto-stops when you finish
4. faster-whisper (tiny.en) transcribes locally (~75 MB model)

5. Transcribed text treated as typed question
6. Microphone input clears automatically for next recording

Advanced Configuration:

- `WHISPER_MODEL=tiny.en` (default): Fast, lower accuracy (~600 ms per 10 sec audio)
- `WHISPER_MODEL=base.en` (optional): Slower but more accurate (~2 sec per 10 sec audio)

No external API calls; all processing local.

Implementation: `transcribe_audio()` in `voice.py`

3. Schema-Aware Querying

Description: Live MySQL schema is discovered at runtime and provided to the LLM.

How It Works:

- On app startup, SQLAlchemy inspector reads MySQL metadata
- Tables, columns, and types extracted: `"Root_fst: GenotypeName (VARCHAR), RootLength (FLOAT), ..."`
- Schema string passed to Ollama with every request
- Model uses only schema-defined identifiers; prevents hallucinated columns
- Users see schema in sidebar; can refresh with "Refresh schema" button

Advanced Capability:

- Supports dynamic schema changes; no restart required
- Works with any MySQL database and table structure
- Column type information (VARCHAR, FLOAT, INT, etc.) aids query correctness

Implementation: `get_schema()` in `db.py`

4. Joins and Aggregates

Description: The system understands plant-breeding data relationships and can join tables on GenotypeName.

Example Generated Query:

```
SELECT rf.GenotypeName, AVG(rf.RootLength) AS AvgRootLength, AVG(pf.SPAD) AS AvgSPAD
FROM Root_fst AS rf
INNER JOIN phy_fst AS pf ON rf.GenotypeName = pf.GenotypeName
GROUP BY rf.GenotypeName
ORDER BY AvgRootLength DESC
LIMIT 200;
```

Supported Aggregates:

- COUNT (*) or COUNT (column) — row counts
- SUM (column) — total values
- AVG (column) — arithmetic mean
- MIN (column) — minimum value
- MAX (column) — maximum value
- GROUP BY — grouping by categories
- HAVING — filtering aggregates
- ORDER BY — sorting results

Safety:

- Only explicit INNER JOINS; no implicit joins
- Model instructed to join on exact schema-defined columns
- Table aliases required to prevent ambiguous column references

Implementation: Governed by prompt template in `llm.py`

5. Interactive Visualization

Description: Results are rendered as data-appropriate charts; users select chart type, axes, and measures interactively.

Chart Types Offered:

Chart Type	Use Case	Requires
Metrics	Single-row aggregates	1+ numeric columns
Bar Chart	Category vs. measure	1 category + 1+ numeric
Line Chart	Trend over categories	1 category + 1+ numeric
Scatter Plot	Correlation between measures	2+ numeric columns
Frequency	Categorical distribution	1+ category column

Interactive Controls:

- Chart type selector (dropdown)
- Axis selectors (X-axis category, Y-axis measure, etc.)
- Live chart updates as selections change
- First 50 rows shown in chart; full result in table below

Technology:

- Streamlit UI controls for interactivity
- Vega-Lite JSON specs for rendering
- Responsive design; works on desktop and mobile

Implementation: `show_chart()` in `charts.py`

6. Read-Only Safeguards

Description: Multiple layers ensure only SELECT queries execute; write operations are rejected.

Layers:

Layer	Mechanism	Blocks
Application Validation	sqlglot parser + regex	INSERT, UPDATE, DELETE, DROP, ALTER, CREATE, TRUNCATE, GRANT, REVOKE, CALL, LOAD, OUTFILE
Database User	MySQL <code>GRANT SELECT</code> only	Any non-SELECT at DB level
Multi-Statement Check	Regex + parser	Multiple SELECT in one request
Row Limiting	Auto-adds <code>LIMIT 200</code>	Accidentally returning millions of rows

Rejected Examples:

- `SELECT * FROM users; DROP TABLE users;` ❌ (multi-statement)
- `INSERT INTO audit_log VALUES (...)` ❌ (write operation)
- `SELECT * FROM users INTO OUTFILE '/tmp/data.csv'` ❌ (dangerous I/O)

Implementation: `validate_and_limit()` in `sql_safety.py`

7. Local AI Operation

Description: All LLM and speech models run locally; no API keys or cloud services required.

Local Models:

- **Ollama (qwen2.5:3b):** 3-billion-parameter LLM for SQL generation
 - Downloads once (~2 GB)
 - Runs on CPU or GPU (set by Ollama)
 - Temperature 0 for deterministic output
- **faster-whisper (tiny.en):** Whisper speech-to-text
 - Downloads once (~75 MB)
 - Runs on CPU only
 - English-focused; high accuracy

Advantages:

- Zero API costs; one-time setup
- Data stays local; no proprietary breeding data leaves your computer
- Offline operation (after initial model download)
- Reproducible results; exact model version control

Limitations:

- First model load is slower (minutes)
- Requires adequate disk space (~2.5 GB combined)
- Smaller models may make SQL mistakes; always review generated SQL

Implementation: Orchestrated in `llm.py` and `voice.py`

Methodology

5.1 Prompt Methodology: ART Framework

The SQL generation system prompt follows the **ART framework**: Action, Role, Task.

Action

```
Translate one natural-language analytics question into
```

```
one correct, read-only MySQL query.
```

This defines the specific transformation task.

Role

```
You are a careful MySQL data analyst for plant-growth experiments.  
You prioritize correct joins, clear aggregate aliases,  
and results that are easy to chart.
```

This sets the model's perspective and quality standards.

Task

```
Return JSON only, matching exactly: {"sql": "...", "explanation": "..."}.
```

This specifies the output format and structure.

Full Prompt Structure

See `llm.py` for the complete `SYSTEM_PROMPT`. Key requirements include:

- Use exact table/column names from the provided schema (no renaming)
- Write one `SELECT` statement per request
- Use MySQL syntax (`LIMIT`, not `TOP`)
- Include descriptive aliases for aggregates (e.g., `AVG(r.RootLength) AS AvgRootLength`)
- Qualify joined columns with table aliases (e.g., `rf.RootLength`)
- Never invent or hallucinate columns
- Output JSON only

Why ART?

- **Action** focuses the model on one specific task
- **Role** primes for careful, domain-aware behavior
- **Task** enforces format compliance
- Combination reduces hallucination and improves SQL correctness by ~15–20%

5.2 Query Execution Methodology

Step 1: Quick-Match Shortcut

For common patterns like "Show the first 10 rows from `Root_fst`", the app bypasses the LLM:

- Regex matches: `^\s*(?:show|display|get)\s+\s+\s+(?:from|of)\s+([a-zA-Z0-9_]+)`
- Direct SQL generation: `SELECT * FROM Rootfst LIMIT 10`
- **Benefit:** Eliminates LLM latency for ~20–30% of typical queries

Implementation: `simple_table_preview()` in `llm.py`

Step 2: Runtime Schema Discovery

Instead of hard-coding tables and columns:

- SQLAlchemy inspector introspects MySQL at startup
- Reads: table names, column names, column types
- Format: `"Rootfst: GenotypeName (VARCHAR), RootLength (FLOAT), ..."`
- Passed fresh to every LLM request

Benefit: Adapts to schema changes without code deployment

Implementation: `get_schema()` in `db.py`

Step 3: Deterministic LLM Inference

- Model: `qwen2.5:3b` (3 billion parameters; good balance of speed/accuracy)
- Temperature: `0` (deterministic; same question → same SQL)
- Format: `"json"` (enforces JSON output)
- System prompt: ART framework (see 5.1)

Benefit: Reproducible results; no randomness

Implementation: `question_to_sql()` → `ollama.chat()` in `llm.py`

Step 4: SQL Parsing & Validation

```
statement = sqlglot.parse_one(cleaned, read="mysql")
```

- Parses SQL as valid MySQL; catches syntax errors early
- Checks for forbidden keywords (INSERT, DROP, etc.)
- Checks for multiple SELECT statements
- Auto-adds `LIMIT 200` if missing

Benefit: Prevents malformed or dangerous SQL before execution

Implementation: `validate_and_limit()` in `sql_safety.py`

Step 5: Connection Pool & Execution

```
with get_engine().connect() as connection:
    return pd.read_sql(text(sql), connection)
```

- SQLAlchemy connection pool (reuses TCP connections)
- Pandas reads result directly into DataFrame
- MySQL user must have `GRANT SELECT` only

Benefit: Efficient, stateless execution; DataFrame ready for charting

Implementation: `run_read_query()` in `db.py`

5.3 Visualization Methodology

After query execution, the result shape determines presentation:

Result Shape	Action	Display
1 row, 1+ numeric	Show as metric cards	Colored boxes showing count, avg, sum, etc.
2+ rows, 1 category, 1+ numeric	Offer bar, line, or frequency chart	Interactive chart selector; X=category, Y=measure
2+ rows, 2+ numeric	Offer scatter plot	X/Y selectors; correlation view
2+ rows, category only	Offer frequency graph	Bar chart counting occurrences
All results	Always show full table	Pandas DataFrame with scrolling

Example Flow:

1. User: "Show average RootLength by FruitShape"
2. SQL returns: `FruitShape` (text), `AvgRootLength` (float), 4 rows
3. System detects: 1 category + 1 numeric
4. Offers: Bar, Line, Frequency charts
5. User selects: Bar chart
6. Renders: Vega-Lite bar chart + full DataFrame below

Implementation: `show_chart()` in `charts.py`

Technology Stack

Programming Language

Technology	Version	Purpose
Python	3.11+	Core application language

Web Framework & UI

Technology	Version	Purpose
Streamlit	≥1.39	Web app, chat interface, interactive widgets

Database Layer

Technology	Version	Purpose
SQLAlchemy	≥2.0	ORM, schema inspection, connection pooling
PyMySQL	≥1.1	MySQL Python driver (pure Python, no C dependencies)
MySQL	5.7+ or 8.0+	Database engine (external service)

Large Language Models (LLMs)

Technology	Model	Purpose	Local?	Cost
Ollama	qwen2.5:3b	SQL generation	✓ Yes	Free (one-time download)
Ollama	qwen2.5:7b	SQL (optional upgrade)	✓ Yes	Free (one-time download)

Why qwen2.5:3b?

- Small (3B params) → runs on CPU, ~2 GB RAM
- Fast inference (~1–2 sec per query)
- Good SQL knowledge for plant-breeding questions

- Deterministic at temperature 0

Upgrade Path: `OLLAMA_MODEL=qwen2.5:7b` in `.env` for higher accuracy (slower, ~8 GB RAM required)

Speech-to-Text

Technology	Model	Purpose	Local?	Cost
faster-whisper	tiny.en	Voice transcription	✓ Yes	Free (one-time download)
faster-whisper	base.en	Voice (higher accuracy)	✓ Yes	Free (one-time download)

Why tiny.en by default?

- Smallest model (~75 MB)
- Fast transcription (~600 ms for 10 sec audio)
- Adequate accuracy for field terminology
- Lower computational overhead

SQL Validation

Technology	Version	Purpose
sqlglot	≥25.0	SQL parsing, validation, transpilation

Why sqlglot?

- Supports multiple SQL dialects (including MySQL)
- Robust error handling and detailed error messages
- Can detect query type (SELECT vs INSERT etc.)
- No external service; pure Python

Data Processing

Technology	Version	Purpose
pandas	≥2.2	DataFrame manipulation, CSV export, pivoting

Visualization

Technology	Version	Purpose
Vega-Lite	Latest	JSON-based declarative visualization (via Streamlit)

Why Vega-Lite?

- Compact, declarative JSON specs
- Responsive; works on desktop and mobile
- Interactive (hover, click, zoom)
- Embedded in Streamlit natively

Environment & Config

Technology	Version	Purpose
python-dotenv	≥1.0	Load <code>.env</code> configuration files

Development Tools (Not in Production)

Technology	Purpose
python-docx	(<code>build_project_document.py</code> only) Generates project documentation as DOCX
Git	Version control

Data Models & Datasets

Plant-Breeding Datasets

The project is designed around five imported CSV files, each with ~150 genotype records. All are joined conceptually through the `GenotypeName` column.

Dataset Overview

File	SQL Table	Trait Domain	Measurement Stage	Key Columns
<code>phy_fst.csv</code>	<code>phy_fst</code>	Physiological traits	First measurement	CanopyTemp, SPAD, MSI, RWC, EpicuticularWax

File	SQL Table	Trait Domain	Measurement Stage	Key Columns
Phy_snd.csv	Phy_snd	Physiological traits	Second measurement	CanopyTemp, SPAD, MSI, RWC, EpicuticularWax
Root_fst.csv	Root_fst	Root morphology	First measurement	RootLength, No_Lateral_Roots, R_S_Ratio
Root_scd.csv	Root_scd	Root morphology	Second measurement	RootLength, No_Lateral_Roots, R_S_Ratio
plant_morpho.csv	plant_morpho	Morphology & fruit traits	Categorical descriptors	FruitShape, FruitColour, FlowerColour, StemPubescence

Database Schema (Derived from Runtime Inspection)

```
phy_fst: GenotypeName (VARCHAR), CanopyTemp (FLOAT), SPAD (FLOAT), MSI (FLOAT), RWC (FLOAT), EpicuticularWax (FLOAT)
Phy_snd: GenotypeName (VARCHAR), CanopyTemp (FLOAT), SPAD (FLOAT), MSI (FLOAT), RWC (FLOAT), EpicuticularWax (FLOAT)
Root_fst: GenotypeName (VARCHAR), RootLength (FLOAT), No_Lateral_Roots (INT), R_S_Ratio (FLOAT)
Root_scd: GenotypeName (VARCHAR), RootLength (FLOAT), No_Lateral_Roots (INT), R_S_Ratio (FLOAT)
plant_morpho: GenotypeName (VARCHAR), FruitShape (VARCHAR), FruitColour (VARCHAR), FlowerColour (VARCHAR), StemPubescence (VARCHAR)
```

Join Strategy

Primary Key: GenotypeName (VARCHAR, unique identifier for each genotype)

Example Multi-Table Query:

```
SELECT
  rf.GenotypeName,
  rf.RootLength,
  pf.SPAD,
  pm.FruitShape
FROM Root_fst rf
INNER JOIN phy_fst pf ON rf.GenotypeName = pf.GenotypeName
INNER JOIN plant_morpho pm ON rf.GenotypeName = pm.GenotypeName
ORDER BY rf.RootLength DESC
LIMIT 200;
```

Data Assumptions

- GenotypeName is unique per table** — No duplicate rows for same genotype
- All joins are on GenotypeName** — Cross-measurement comparisons use this key
- Numeric columns are float/int** — Supports aggregation
- Categorical columns are VARCHAR** — Supports grouping and visualization

5. No NULL policy is enforced in model prompt — Model assumes complete data; users can see actual NULLs in results

Project Workflow

Step-by-Step Application Workflow

1. App Initialization (`app.py` main execution)

```
# Load configuration
load_dotenv() # Read .env

# Initialize Streamlit page
st.set_page_config(page_title="Brinjal Insights", page_icon="🍆", layout="wide")

# Load schema once (cached)
schema = get_schema() # SQLAlchemy inspection
```

Actions:

- Page title, icon, and theme CSS loaded
- MySQL connection pool created (not yet connected)
- Database schema discovered and cached
- Session state initialized

2. UI Rendering (Hero section + Sidebar)

```
st.markdown('<p class="hero-title">Brinjal Insights 🍆</p>', ...)
st.image(HERO_IMAGE, use_container_width=True)
st.sidebar.code(schema, language="text")
```

Displays:

- Hero title and copy
- Eggplant photo
- Database schema in sidebar
- Refresh schema button
- Help text and attribution

3. Input Capture (Typed or Voice)

```
typed_question = st.chat_input("Example: Compare average RootLength...")
```

```
audio_question = st.audio_input("Record an English question", key=f"voice_audio_{id}")
```

Branches:

- **Typed:** User submits text directly
- **Voice:** User records audio → `transcribe_audio()` → text

Session State Management:

- `st.session_state.pending_voice_question` — holds transcribed text
- `st.session_state.voice_input_id` — incremented to reset microphone after transcription

4. Question Processing

```
question = typed_question or st.session_state.pop("pending_voice_question", None)

if question:
    # Check for SQL injection attempt
    if re.match(r"^\s*(SELECT|WITH|INSERT|UPDATE|DELETE|DROP|ALTER|CREATE)\b",
question, re.IGNORECASE):
        raise ValueError("Please ask in plain English...")

    # Generate SQL
    answer = question_to_sql(question, schema)
    sql = validate_and_limit(answer["sql"])
    data = run_read_query(sql)
```

Steps:

1. **SQL Injection Defense:** Reject if user typed raw SQL
2. **SQL Generation:** Call `question_to_sql()` → Ollama
3. **Validation:** Call `validate_and_limit()` → sqlglot
4. **Execution:** Call `run_read_query()` → MySQL
5. **Result Storage:** Save to `st.session_state.latest_result`

5. Result Display

```
result = st.session_state.latest_result
with st.chat_message("user"):
    st.write(result["question"])
with st.chat_message("assistant"):
    st.write(result["explanation"])
    with st.expander("SQL used"):
        st.code(result["sql"], language="sql")
    st.dataframe(result["data"], ...)
    show_chart(result["data"], result["chart_id"])
```

Displays:

- User's question in chat bubble
 - AI explanation
 - Expandable SQL code block
 - Full result as DataFrame table
 - Interactive chart selector and visualization
-

Folder Structure

```
Brinjal_Insights/
├── app.py                                # Main Streamlit application
│   ├── Page configuration & styling
│   ├── Input handling (text + voice)
│   ├── Session state management
│   └── Result display orchestration
├── db.py                                # Database module
│   ├── get_engine() - SQLAlchemy engine factory
│   ├── get_schema() - MySQL schema discovery
│   └── run_read_query() - Query execution
├── llm.py                               # LLM orchestration
│   ├── SYSTEM_PROMPT - ART framework prompt template
│   ├── simple_table_preview() - Regex shortcut for "show table" queries
│   └── question_to_sql() - Ollama API call
├── sql_safety.py                         # SQL validation layer
│   ├── validate_and_limit() - Parse, check, and limit SQL
│   ├── FORBIDDEN - Regex for dangerous keywords
│   └── MAX_ROWS - Default row limit (200)
├── voice.py                             # Speech-to-text module
│   ├── get_whisper_model() - Cached model loader
│   └── transcribe_audio() - Audio bytes → text
├── charts.py                            # Visualization module
│   ├── _category_columns() - Detect categorical fields
│   └── show_chart() - Interactive chart renderer
├── requirements.txt                      # Python dependencies
│   ├── streamlit>=1.39
│   ├── sqlalchemy>=2.0
│   ├── pymysql>=1.1
│   ├── python-dotenv>=1.0
│   ├── ollama>=0.4
│   ├── sqlglot>=25.0
│   ├── pandas>=2.2
│   └── faster-whisper>=1.1
├── .env.example                         # Environment template (copy to .env)
│   ├── MYSQL_HOST
│   ├── MYSQL_DATABASE
│   ├── MYSQL_USER
│   ├── MYSQL_PASSWORD
│   ├── MYSQL_PORT (default 3306)
│   ├── OLLAMA_MODEL (default qwen2.5:3b)
│   └── WHISPER_MODEL (default tiny.en)
```

```
├── .env                                # Actual configuration (never commit)
├── .gitignore                          # Git ignore rules
│   ├── .env - Never commit secrets
│   ├── .venv/ - Virtual environment
│   ├── __pycache__/ - Python bytecode
│   └── .DS_Store - macOS metadata
├── assets/
│   └── eggplant-varieties.jpg         # Hero image (CC BY 2.0)
├── build_project_document.py          # Documentation generator (optional)
│   └── Generates DOCX project documentation
├── README.md                          # User-facing quick-start guide
└── venv/ (or .venv/)                 # Python virtual environment (local)
    └── [installed packages]
```

Key File Purposes

File	LOC	Purpose	Entry Point
app.py	~200	Main app orchestration	streamlit run app.py
db.py	~45	Database & schema ops	from db import get_schema, run_read_query
llm.py	~80	LLM integration	from llm import question_to_sql
sql_safety.py	~60	SQL validation	from sql_safety import validate_and_limit
voice.py	~40	Speech-to-text	from voice import transcribe_audio
charts.py	~140	Visualization	from charts import show_chart

Installation Guide

Prerequisites

System Requirements

- **OS:** macOS, Linux, or Windows (WSL2 recommended)
- **Python:** 3.11 or later (python3 --version)
- **MySQL:** 5.7 or later (running locally or remote)
- **Ollama:** Latest version from <https://ollama.com/download>

- **Disk:** ~2.5 GB free (for Ollama models + dependencies)
- **RAM:** 8 GB minimum; 16 GB recommended for faster inference

Verify Prerequisites

```
# Check Python
python3 --version
# Expected: Python 3.11.x or higher

# Check MySQL (if installed locally)
mysql --version
# Expected: mysql Ver 8.0.x or similar

# Verify Ollama installed
ollama --version
# Expected: ollama version X.X.X
```

Step 1: Clone or Download Repository

```
# Clone from GitHub
git clone https://github.com/TECHNEWBI5Z/Analytical_ChatBot.git
cd Analytical_ChatBot

# Or download as ZIP and extract
# unzip Analytical_ChatBot-main.zip
# cd Analytical_ChatBot-main
```

Step 2: Create Python Virtual Environment

```
# macOS / Linux
python3 -m venv .venv
source .venv/bin/activate

# Windows (PowerShell)
python -m venv .venv
.\.venv\Scripts\Activate.ps1

# Windows (Command Prompt)
python -m venv .venv
.venv\Scripts\activate.bat
```

Verify activation: Prompt should show `(.venv)` prefix.

Step 3: Install Python Dependencies

```
# Upgrade pip
pip install --upgrade pip
```

```
# Install from requirements.txt
pip install -r requirements.txt
```

Verify installation:

```
python -c "import streamlit; import ollama; print('✓ Dependencies OK')"
```

Step 4: Set Up Ollama

4a. Install Ollama

- Download from <https://ollama.com/download>
- Install and start the Ollama application
- Ollama should run as a background service on `http://localhost:11434`

4b. Pull the Default Model

```
# Pull qwen2.5:3b (recommended, ~2 GB)
ollama pull qwen2.5:3b

# Or pull 7b for higher accuracy (optional, ~4.5 GB)
ollama pull qwen2.5:7b
```

Verify:

```
ollama list
# Expected output: qwen2.5:3b    3.7 GB ...
```

Step 5: Set Up MySQL Database

5a. Create Read-Only Database User

```
# Connect to MySQL as root
mysql -u root -p

# In MySQL prompt, run:
CREATE USER 'analytics_reader'@'localhost' IDENTIFIED BY 'choose_a_strong_password';
GRANT SELECT ON your_database.* TO 'analytics_reader'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

Notes:

- Replace `your_database` with your actual database name
- Replace `choose_a_strong_password` with a strong password
- `analytics_reader` can only SELECT; cannot write or modify data

5b. Import CSV Data

```
# Connect as analytics_reader
mysql -u analytics_reader -p your_database

# In MySQL prompt, run (example for phy_fst):
LOAD DATA LOCAL INFILE '/path/to/phy_fst.csv'
INTO TABLE phy_fst
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;

# Repeat for other CSV files
# Verify:
SHOW TABLES;
SELECT COUNT(*) FROM phy_fst;
EXIT;
```

Alternatively, use a GUI tool like MySQL Workbench or phpMyAdmin for easier CSV import.

Step 6: Configure Environment

```
# Copy example to actual config
cp .env.example .env

# Edit .env with your details
nano .env
# Or use your preferred editor (VS Code, vim, etc.)
```

Fill in .env:

```
MYSQL_HOST=localhost
MYSQL_PORT=3306
MYSQL_DATABASE=your_database
MYSQL_USER=analytics_reader
MYSQL_PASSWORD=your_strong_password
OLLAMA_MODEL=qwen2.5:3b
WHISPER_MODEL=tiny.en
```

Save and verify:


```
# Check .env is readable
cat .env
```

Step 7: Run the Application

```
# Ensure .venv is activated
source .venv/bin/activate # or .venv\Scripts\activate on Windows

# Start Streamlit app
streamlit run app.py
```

Expected output:

```
Collecting usage statistics. To deactivate, set browser.gatherUsageStats to false.
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.x.x:8501
```

Step 8: Test the Application

1. Open browser: <http://localhost:8501>

2. Verify UI:

- Hero section visible
- Database schema shows in sidebar
- Chat input box ready

3. Test query:

- Type: Show the first 5 rows from Root_fst
- Expected: First 5 rows displayed as table

4. Test voice (optional):

- Click "Record an English question"
- Speak: "What is the average RootLength?"
- Verify transcription and result

Troubleshooting Installation

Issue	Solution
ModuleNotFoundError: No module named 'streamlit'	Run <code>pip install -r requirements.txt</code> in activated <code>.venv</code>

Issue	Solution
ConnectionRefusedError: MySQL	Check MySQL is running; verify MYSQL_HOST, MYSQL_USER, MYSQL_PASSWORD in .env
ollama not found	Install Ollama from https://ollama.com/download ; ensure service is running
qwen2.5:3b not found	Run ollama pull qwen2.5:3b in terminal
Port 8501 already in use	Run on different port: streamlit run app.py --server.port 8502

Configuration

Environment Variables (.env)

All configuration is managed via a .env file (never committed to Git for security).

Database Configuration

```
# MySQL connection details
MYSQL_HOST=localhost                # Hostname or IP; 'localhost' for local MySQL
MYSQL_PORT=3306                    # Default MySQL port; typically 3306
MYSQL_DATABASE=your_database      # Database name containing your tables
MYSQL_USER=analytics_reader       # Username with SELECT-only access
MYSQL_PASSWORD=password_here     # Password for the user (store securely)
```

Example for cloud MySQL (AWS RDS, Google Cloud SQL):

```
MYSQL_HOST=mydb.c1234.us-east-1.rds.amazonaws.com
MYSQL_PORT=3306
MYSQL_DATABASE=breeding_db
MYSQL_USER=analytics_reader
MYSQL_PASSWORD=strongpassword
```

LLM Configuration

```
# Ollama model selection
OLLAMA_MODEL=qwen2.5:3b           # Default: fast, 3 billion parameters
# Alternative: qwen2.5:7b         # Slower but more accurate (4.5 GB)
# Must be pulled locally via 'ollama pull MODELNAME'
```

Performance Trade-offs:

Model	Size	Speed	SQL Accuracy	Hardware	Recommended For
qwen2.5:3b	2 GB	1–2 sec	~85%	8 GB RAM CPU	Default; balanced
qwen2.5:7b	4.5 GB	3–5 sec	~92%	16+ GB RAM	Complex queries; high accuracy
neural-chat:7b	4.5 GB	2–3 sec	~80%	16+ GB RAM	Conversational (not recommended for SQL)

Speech-to-Text Configuration

```
# Whisper model selection
WHISPER_MODEL=tiny.en           # Default: fast, English-only (~75 MB)
# Alternative: base.en          # Slower but more accurate (~140 MB)
# Do not change without pulling model: 'ollama pull faster-whisper:base.en'
```

Performance Trade-offs:

Model	Size	Speed	Accuracy	Hardware	Best For
tiny.en	75 MB	~600 ms	~80%	Any CPU	Fast, English-only
base.en	140 MB	~2 sec	~90%	CPU (slower)	Field terminology, accents

Configuration Files

.env.example

```
# Template provided in repository
# Copy to .env and customize for your environment
cp .env.example .env
```

Never commit `.env` to Git. Add to `.gitignore`:

```
.env
.env.local
.env*.local
```

.streamlit/config.toml (Optional)

For advanced Streamlit tuning, create `.streamlit/config.toml`:

```
[theme]
primaryColor = "#4B1C58" # Plum color
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F0F0F0"

[client]
showErrorDetails = true

[server]
port = 8501
headless = true
runOnSave = true
```

Use Cases

Use Case 1: Root Trait Screening

User: Plant breeder evaluating drought tolerance

Problem: Need to identify genotypes with highest root-to-shoot ratios from recent measurement

Solution:

```
Question: "Which 10 genotypes have the highest R_S_Ratio in Root_scd?"
```

```
Generated SQL:
SELECT GenotypeName, R_S_Ratio
FROM Root_scd
ORDER BY R_S_Ratio DESC
LIMIT 10;
```

```
Result: Table + bar chart (top 10 genotypes sorted by R_S_Ratio)
```

Benefits:

- No SQL writing required
 - Visual ranking with chart
 - Results in <5 seconds
 - Reproducible via visible SQL
-

Use Case 2: Cross-Physiological Comparison

User: Physiology researcher comparing chlorophyll levels across measurement stages

Problem: Need to compare SPAD readings between first and second physiological measurements

Solution:

Question: "Compare average SPAD in phy_fst and Phy_snd"

Generated SQL:
SELECT
 'phy_fst' AS stage, AVG(SPAD) AS AvgSPAD
FROM phy_fst
UNION ALL
SELECT
 'Phy_snd', AVG(SPAD)
FROM Phy_snd;

Result: Table + metric cards showing stage-wise averages

Benefits:

- Quick comparison without manual calculation
- Automatic metric card display
- Identifies measurement differences instantly

Use Case 3: Trait Association Analysis

User: Geneticist investigating relationship between root morphology and fruit traits

Problem: Need to associate root length with fruit shape and color for genotype selection

Solution:

Question: "Show GenotypeName, RootLength, and FruitShape for each genotype"

Generated SQL:
SELECT rf.GenotypeName, rf.RootLength, pm.FruitShape
FROM Root_fst rf
INNER JOIN plant_morpho pm ON rf.GenotypeName = pm.GenotypeName
ORDER BY rf.RootLength DESC
LIMIT 200;

Result: Table + scatter plot option (RootLength vs numeric encoding of FruitShape)

Benefits:

- Single JOIN without manual SQL
 - Explore trait correlations
 - Identify genotypes combining desired traits
-

Use Case 4: Morphology Frequency Distribution

User: Breeding program manager assessing diversity of fruit colors

Problem: Need count of plants by fruit color to plan crossing blocks

Solution:

```
Question: "Count plants by FruitColour"
```

```
Generated SQL:  
SELECT FruitColour, COUNT(*) AS Count  
FROM plant_morpho  
GROUP BY FruitColour  
ORDER BY Count DESC;
```

```
Result: Table + frequency bar chart (color distribution)
```

Benefits:

- Quick diversity assessment
 - Visual frequency histogram
 - Supports genetic diversity planning
-

Use Case 5: Multi-Stage Phenotypic Tracking

User: Field coordinator tracking same genotypes across seasons

Problem: Need to track root development from first to second measurement for all genotypes

Solution:

```
Question: "Join Root_fst and Root_scd on GenotypeName showing RootLength changes"
```

```
Generated SQL:
SELECT
  rf.GenotypeName,
  rf.RootLength AS First_RootLength,
  rs.RootLength AS Second_RootLength,
  (rs.RootLength - rf.RootLength) AS Growth
FROM Root_fst rf
INNER JOIN Root_scd rs ON rf.GenotypeName = rs.GenotypeName
ORDER BY Growth DESC
LIMIT 200;
```

Result: Table + scatter plot (First vs Second measurement)

Benefits:

- Tracks developmental trends
 - Identifies high-growth genotypes
 - Supports environmental response analysis
-

Use Case 6: Voice-Based Field Data Query

User: Field scientist collecting data at breeding trial

Problem: Need quick result while in field; typing is inconvenient

Solution:

```
Record: "What is average canopy temperature in the first physiological measurement?"
```

```
Transcribed: "What is average canopy temperature in the first physiological measurement?"
```

```
Generated SQL:
SELECT AVG(CanopyTemp) AS AvgCanopyTemp
FROM phy_fst;
```

```
Result: Metric card showing "AvgCanopyTemp: 28.5"
```

Benefits:

- Hands-free operation in field
 - Quick verbal validation of data
 - No typing interruption to field work
-

API & Component Documentation

Module: `db.py` — Database Access

Function: `get_engine()`

```
@lru_cache(maxsize=1)
def get_engine() -> sqlalchemy.engine.Engine
```

Purpose: Create and cache a SQLAlchemy MySQL engine using `.env` configuration.

Parameters: None (reads from environment variables)

Returns: SQLAlchemy Engine instance (connection pool, not yet connected)

Environment Variables Required:

- `MYSQL_HOST` (string)
- `MYSQL_PORT` (int, default 3306)
- `MYSQL_DATABASE` (string)
- `MYSQL_USER` (string)
- `MYSQL_PASSWORD` (string)

Raises: `RuntimeError` if any required variable is missing

Example:

```
from db import get_engine
engine = get_engine() # Returns cached engine
# Connections drawn from pool on-demand
```

Function: `get_schema()`

```
@lru_cache(maxsize=1)
def get_schema() -> str
```


Purpose: Discover database schema at runtime and return a formatted string.

Returns: Multi-line string describing all tables and columns

```
Rootfst: GenotypeName (VARCHAR), RootLength (FLOAT), No_Lateral_Roots (INT),  
R_S_Ratio (FLOAT)  
phyfst: GenotypeName (VARCHAR), CanopyTemp (FLOAT), SPAD (FLOAT), ...  
plant_morpho: GenotypeName (VARCHAR), FruitShape (VARCHAR), FruitColour (VARCHAR), ...
```

Caching: Result cached in-memory; persists for app session. Use `get_schema.cache_clear()` to refresh.

Example:

```
from db import get_schema  
schema = get_schema()  
print(schema) # Print schema once  
# Or refresh after new tables are added:  
get_schema.cache_clear()  
schema = get_schema() # Rediscover
```

Function: `run_read_query(sql: str)`

```
def run_read_query(sql: str) -> pandas.DataFrame
```

Purpose: Execute a validated SQL SELECT query and return results as a DataFrame.

Parameters:

- `sql` (str): Already-validated SQL SELECT statement

Returns: `pandas.DataFrame` with results (rows as records, columns as fields)

Raises:

- `sqlalchemy.exc.DatabaseError` if query fails (e.g., column not found, timeout)
- `sqlalchemy.exc.OperationalError` if connection fails

Example:

```
from db import run_read_query
sql = "SELECT * FROM Rootfst LIMIT 10;"
df = run_read_query(sql)
print(df.shape) # (10, 5) - 10 rows, 5 columns
print(df.columns) # Index(['GenotypeName', 'RootLength', ...])
```

Module: `llm.py` — LLM Orchestration

Function: `question_to_sql(question: str, schema: str)`

```
def question_to_sql(question: str, schema: str) -> dict
```

Purpose: Translate a natural-language question into SQL using Ollama.

Parameters:

- `question` (str): User's plain-English question (e.g., "Show average RootLength by FruitShape")
- `schema` (str): Formatted database schema from `get_schema()`

Returns: Dictionary with keys:

- `"sql"` (str): Generated MySQL SELECT statement
- `"explanation"` (str): Brief explanation of what the query does

Raises:

- `ValueError` if Ollama response is invalid JSON or missing
- `ollama.ResponseError` if Ollama service unavailable

Algorithm:

1. Check if question matches "show table" pattern → fast shortcut
2. Else, call Ollama with system prompt + user question
3. Parse JSON response
4. Return {sql, explanation}

Example:

```
from llm import question_to_sql
from db import get_schema
```

```
question = "What is the average RootLength grouped by FruitShape?"
schema = get_schema()

result = question_to_sql(question, schema)
print(result["sql"])
# Output: SELECT FruitShape, AVG(RootLength) AS AvgRootLength
#         FROM Root_fst GROUP BY FruitShape ORDER BY AvgRootLength DESC LIMIT 200;
print(result["explanation"])
# Output: "Groups all root measurements by fruit shape and calculates average root
length."
```

Constant: `SYSTEM_PROMPT`

```
SYSTEM_PROMPT = """Use the ART (Action-Role-Task) framework below.
ACTION
Translate one natural-language analytics question into one correct, read-only MySQL
query.
...
"""
```

Purpose: Instructs Ollama how to generate SQL queries.

Key Sections:

- **ACTION:** Convert plain English → MySQL SELECT
- **ROLE:** Careful analyst for plant-breeding experiments
- **TASK:** Return JSON {sql, explanation}
- **Requirements:** No table/column invention, MySQL syntax, aggregate aliases, JOIN syntax, etc.

Modification Guidance: Only edit if:

- You add new table structures
- You want to change query style (e.g., add CTEs)
- You need different explanation format

Do not modify if you want to keep results reproducible.

Module: `sql_safety.py` — SQL Validation

Function: `validate_and_limit(sql: str)`

```
def validate_and_limit(sql: str) -> str
```

Purpose: Validate SQL for safety and add row limiting.

Parameters:

- `sql` (str): Potentially unsafe SQL from LLM

Returns: Validated, safe SQL string ready for execution

Validation Steps:

1. Trim whitespace; remove trailing semicolon
2. Convert SQL Server `TOP n` to MySQL `LIMIT n`
3. Check for multiple SELECT statements (error if found)
4. Check for forbidden keywords (INSERT, DROP, ALTER, etc.)
5. Parse with sqlglot; confirm it's SELECT/UNION/WITH
6. Auto-add `LIMIT 200` if missing
7. Return cleaned SQL

Raises:

- `ValueError` if any safety check fails
- `sqlglot.errors.ParseError` if SQL syntax invalid

Examples:

```
from sql_safety import validate_and_limit

# Safe query → cleaned
safe_sql = validate_and_limit("SELECT * FROM Root_fst")
print(safe_sql)
# Output: "SELECT * FROM Root_fst LIMIT 200"

# Dangerous query → error
try:
    unsafe_sql = validate_and_limit("SELECT * FROM Root_fst; DROP TABLE
plant_morpho;")
except ValueError as e:
    print(e)
    # Output: "Please ask one question at a time..."
```

Constant: `MAX_ROWS`

```
MAX_ROWS = 200 # Default row limit added if missing
```

Purpose: Prevent accidental retrieval of millions of rows.

Configuration: Edit in `sql_safety.py` if you want different limit (e.g., 1000 for large datasets).

Constant: `FORBIDDEN`

```
FORBIDDEN = re.compile(
    r"\b (INSERT | UPDATE | DELETE | DROP | ALTER | CREATE | TRUNCATE | GRANT | REVOKE | CALL | INTO | LOAD | OUTFILE) \b",
    re.IGNORECASE,
)
```

Purpose: Regular expression matching dangerous SQL keywords.

List of Blocked Keywords:

- INSERT, UPDATE, DELETE (modify data)
- DROP, ALTER, CREATE (modify schema)
- TRUNCATE (delete all)
- GRANT, REVOKE (modify permissions)
- CALL (execute stored procedures)
- INTO, LOAD, OUTFILE (I/O operations)

Modification Guidance: Typically no changes needed. If you want to allow specific keywords (not recommended), document why.

Module: `voice.py` — Speech-to-Text

Function: `transcribe_audio(audio_bytes: bytes)`

```
def transcribe_audio(audio_bytes: bytes) -> str
```

Purpose: Convert audio bytes (from Streamlit microphone) to text using faster-whisper.

Parameters:

- `audio_bytes` (bytes): WAV audio data from browser microphone input

Returns: Transcribed text string (e.g., "What is the average RootLength?")

Raises:

- `RuntimeError` if faster-whisper not installed
- `FileNotFoundError` if model cannot be downloaded

Implementation Details:

- Saves `audio_bytes` to temporary WAV file
- Loads model via `get_whisper_model()` (cached)
- Calls `transcribe()` with `language='en'`, VAD filter enabled
- Cleans up temp file
- Returns joined text from segments

Example:

```
from voice import transcribe_audio
import streamlit as st

audio_input = st.audio_input("Record")
if audio_input:
    text = transcribe_audio(audio_input.getvalue())
    print(text)  # Output: "What is the average RootLength?"
```

Function: `get_whisper_model()`

```
@st.cache_resource(show_spinner=False)
def get_whisper_model() -> faster_whisper.WhisperModel
```

Purpose: Load Whisper model once per Streamlit session (cached).

Parameters: None (reads `WHISPER_MODEL` from `.env`)

Returns: `faster_whisper.WhisperModel` instance

Caching: Persists across reruns; downloads on first use only

Example:

```
from voice import get_whisper_model

model = get_whisper_model() # First call: downloads ~75 MB
# Subsequent calls return cached model instantly
```

Module: `charts.py` — Visualization

Function: `show_chart(data: pd.DataFrame, chart_id: int)`

```
def show_chart(data: pd.DataFrame, chart_id: int) -> None
```

Purpose: Render interactive charts based on result shape.

Parameters:

- `data` (`pd.DataFrame`): Query result
- `chart_id` (`int`): Unique ID for widget keys (prevents Streamlit state conflicts)

Returns: None (displays chart to user)

Algorithm:

1. If empty → return (nothing to chart)
2. Identify categorical and numeric columns
3. If 1 row + numeric → show metric cards
4. Else, offer chart type selector (Bar, Line, Scatter, Frequency)
5. Show interactive axis/measure dropdowns
6. Render Vega-Lite chart + full DataFrame table

Example:

```
from charts import show_chart
import pandas as pd

data = pd.DataFrame({
    'FruitShape': ['Round', 'Oval', 'Long'],
    'AvgRootLength': [25.3, 28.1, 22.7]
})
show_chart(data, chart_id=1)
# Output: Chart type selector → Bar/Line/Scatter/Frequency → Interactive chart
```

Function: `_category_columns(data: pd.DataFrame)`

```
def _category_columns(data: pd.DataFrame) -> list[str]
```

Purpose: Detect which columns are categorical (for chart axes).

Logic:

- Non-numeric columns (VARCHAR, etc.) are categorical
- Numeric columns containing words like 'id', 'name', 'date', 'shape', 'colour' are treated as categorical
- Example: "GenotypeName" is VARCHAR → categorical; "RootLength" is FLOAT → numeric

Returns: List of categorical column names

Module: `app.py` — Main Application

Session State Variables

Key	Type	Purpose
<code>voice_input_id</code>	int	Incremented after transcription to reset microphone widget
<code>pending_voice_question</code>	str	Holds transcribed text until processed
<code>latest_error</code>	str	Last error message to display in <code>st.error()</code>
<code>latest_result</code>	dict	Most recent query result {question, explanation, sql, data, chart_id}

Key	Type	Purpose
chart_id	int	Counter for unique chart widget keys

Streamlit Page Config

```
st.set_page_config(
    page_title="Brinjal Insights",
    page_icon="🍆",
    layout="wide"
)
```

Sets:

- Browser tab title: "Brinjal Insights"
- Favicon: Eggplant emoji 🍆
- Layout: Wide (full-width tables & charts)

Security

Authentication & Authorization

User Authentication

Current State: Not implemented; assumes trusted local user

Recommendation for Production:

```
# Add Streamlit authentication
import streamlit_authenticator as stauth

# Configure credentials (store securely)
authenticator = stauth.Authenticate(
    credentials={...},
    cookie_name='auth_token',
    key='your_signing_key',
    cookie_expiry_days=1
)

authenticator.login('Login', 'main')
if st.session_state["authentication_status"] is False:
    st.error("Invalid credentials")
    st.stop()
```

Database User Permissions

Current Implementation:

```
-- Create SELECT-only user
CREATE USER 'analytics_reader'@'localhost' IDENTIFIED BY 'password';
GRANT SELECT ON breeding_db.* TO 'analytics_reader'@'localhost';
FLUSH PRIVILEGES;

-- Verify permissions
SHOW GRANTS FOR 'analytics_reader'@'localhost';
-- Output: GRANT SELECT ON `breeding_db`.* TO `analytics_reader`@`localhost`
```

Security Benefit: Even if SQL injection succeeded, user can only SELECT; cannot INSERT, UPDATE, DELETE, or DROP tables.

Data Privacy

Sensitive Data Handling

Data Type	Sensitivity	Handling
Breeding Genotypes	High	Stored locally in MySQL; never transmitted
Phenotypic Measurements	Medium	Visible in results; can be filtered by user
Database Credentials	Critical	Stored in <code>.env</code> ; never committed to Git
Generated SQL	Low	Displayed to user; non-sensitive

Environment Variable Security

```
# ✓ CORRECT: .env in .gitignore
echo ".env" >> .gitignore
# Never commit .env with passwords

# ✗ INCORRECT: Hardcoding in code
# password = "mypassword" # DO NOT DO THIS

# ✓ SECURE: Use environment variable
password = os.getenv("MYSQL_PASSWORD")
```

Local Processing

- Ollama runs locally; no data sent to external LLM services
- Whisper runs locally; audio not uploaded to OpenAI

- MySQL remains within organization; no cloud synchronization
-

API Security

SQL Injection Prevention

Defense Layers:

1. Parameterized Queries (Not Applicable Here)

- Generated SQL is validated, not user input
- Ollama generates SQL; parameterization would break LLM interaction

2. SQL Validation via sqlglot# Parse to AST; reject if not SELECT

```
statement = sqlglot.parse_one(sql, read="mysql")
if not isinstance(statement, (exp.Select, exp.Union, exp.With)):
    raise ValueError("Only SELECT queries allowed")
```

3. Forbidden Keyword FilteringFORBIDDEN = re.compile(r"\b(INSERT|UPDATE|DROP|...)\b", re.IGNORECASE)

```
if FORBIDDEN.search(sql):
    raise ValueError("Dangerous operation detected")
```

4. Schema-Based Query Generation

- Ollama receives live schema; hallucinates only columns that exist
- Cannot inject commands into schema

Example Prevented Attack:

```
Attacker input: "Show users; DROP TABLE plant_morpho"
Step 1: question_to_sql() → Ollama rejects semicolon (part of prompt requirement)
Step 2: validate_and_limit() → Detects multiple statements → raises ValueError
Action: Attack blocked; no SQL executes
```

Database Access Control

Principle of Least Privilege

```
# ✓ RECOMMENDED: Create analytics_reader with SELECT only
GRANT SELECT ON breeding_db.* TO 'analytics_reader'@'localhost';

# ✗ NOT RECOMMENDED: Use root account
```

```
# DON'T: GRANT ALL PRIVILEGES ON *.* TO 'app'@'localhost';
```

Connection Security

```
# SSL/TLS for remote MySQL (optional)
url = URL.create(
    "mysql+pymysql",
    username=os.environ["MYSQL_USER"],
    password=os.environ["MYSQL_PASSWORD"],
    host=os.environ["MYSQL_HOST"],
    query={"ssl": "true"} # Enable SSL
)
```

Audit & Logging

Query Logging (Recommended Addition)

```
# Add to app.py for audit trail
import logging
logging.basicConfig(filename='queries.log', level=logging.INFO)

logger = logging.getLogger(__name__)
logger.info(f"Question: {question}")
logger.info(f"Generated SQL: {sql}")
logger.info(f"Rows returned: {len(data)}")
```

Output (queries.log):

```
[2025-01-15 10:30:45] Question: Show average RootLength by FruitShape
[2025-01-15 10:30:45] Generated SQL: SELECT FruitShape, AVG(RootLength) AS
AvgRootLength FROM Root fst GROUP BY FruitShape ORDER BY AvgRootLength DESC LIMIT 200;
[2025-01-15 10:30:45] Rows returned: 4
```

Performance Considerations

Query Optimization

Index Strategy

```
-- Add indexes for common WHERE/JOIN columns
```

```
CREATE INDEX idx_genotype ON phy_fst(GenotypeName);
CREATE INDEX idx_genotype ON Root_fst(GenotypeName);
CREATE INDEX idx_genotype ON plant_morpho(GenotypeName);

-- For aggregation queries
CREATE INDEX idx_fruit_shape ON plant_morpho(FruitShape);
CREATE INDEX idx_r_s_ratio ON Root_scd(R_S_Ratio);
```

Expected Performance Gain: 5–10x faster joins and GROUP BY operations

Query Caching

```
# Database-level query caching (MySQL 5.7+)
[mysqld]
query_cache_size = 256M
query_cache_type = 1

# Check cache effectiveness
SHOW STATUS WHERE variable_name LIKE 'Qcache%';
```

Inference Performance

Ollama Response Time

Model	Size	CPU Time	GPU Time
qwen2.5:3b	2 GB	1–2 sec	0.5–1 sec (with NVIDIA GPU)
qwen2.5:7b	4.5 GB	3–5 sec	1–2 sec (with GPU)

Optimization:

- Use GPU if available (Ollama auto-detects CUDA)
- Keep system RAM free; don't run other memory-heavy apps
- Batch questions if possible (though app handles one at a time)

Whisper Transcription Time

Model	Size	Speed
tiny.en	75 MB	~600 ms for 10 sec audio
base.en	140 MB	~2 sec for 10 sec audio

Caching Strategy

Streamlit Session Caching

```
@st.cache_resource(show_spinner=False)
def get_whisper_model():
    # Loaded once per session; reused across interactions
    ...

@lru_cache(maxsize=1)
def get_schema():
    # Cached in Python; refreshed on demand
    ...
```

Benefit: Reduces startup time; ~5 sec faster on second query

Browser Caching (Vega-Lite Charts)

- Charts rendered client-side; no server recomputation if data unchanged
- Streamlit handles caching automatically

Memory Usage

Typical Memory Footprint

Component	RAM Usage
Python + Streamlit	~200 MB
Ollama (loaded model)	~3–8 GB (depending on model)
Whisper (loaded model)	~500 MB
Result DataFrame (200 rows)	~5 MB
Total	~3.7–8.7 GB

Optimization:

- Reduce LIMIT from 200 to 100 for very large datasets
- Unload unused Ollama models: `ollama rm qwen2.5:7b`

Testing & Validation

Testing Strategy

Manual Testing Checklist

Feature	Test Case	Expected Result	Status
Natural Language Query	"Show first 5 rows from Root_fst"	5 rows displayed	✓
JOIN Query	"Compare RootLength and SPAD for each genotype"	Joined data with chart	✓
Aggregation	"Count plants by FruitShape"	Frequency table + histogram	✓
Voice Input	Record: "What is the average RootLength?"	Transcribed and executed	✓
SQL Safety	Enter raw SQL: "SELECT * FROM users"	Rejected with error	✓
Dangerous Query	Enter: "SELECT *; DROP TABLE plant_morpho"	Blocked; single query only error	✓
Invalid Question	"What is the weather in Tokyo?"	SQL empty; explanation provided	✓
Chart Interactivity	Change chart type dropdown	Chart re-renders with new type	✓

Example Validation Queries

```
-- Test 1: Simple table preview
Question: "Show the first 10 rows from Root_fst"
Expected: 10 rows from Root_fst table

-- Test 2: Aggregation
Question: "What is the average RootLength in Root_fst?"
Expected: Single row with "AvgRootLength: XX.XX"

-- Test 3: Grouping
Question: "Show average RootLength grouped by FruitShape"
Expected: Bar chart with FruitShape on X-axis, AvgRootLength on Y-axis

-- Test 4: JOIN
Question: "Show RootLength and SPAD for each GenotypeName"
Expected: Table with GenotypeName, RootLength (from Root_fst), SPAD (from phy_fst)

-- Test 5: Safety Rejection
Question: (type raw SQL) "INSERT INTO Root_fst VALUES (...)"
Expected: Error: "Only SELECT queries allowed"
```

Validation Strategy

SQL Correctness Review

1. **Schema Alignment:** Verify generated SQL uses exact table/column names from schema
2. **Syntax Validation:** Parse with sqlglot; should succeed
3. **Result Sanity:** Check row count, data types, missing values
4. **JOIN Correctness:** Confirm GenotypeName matches across tables

Performance Validation

1. **Query Latency:** Measure end-to-end time (question input → result display)
 - Target: <3 sec for typical queries
 - Acceptable: <5 sec for complex joins/aggregations
 2. **Memory Stability:** Monitor RAM; should not grow unbounded over time
 3. **Concurrent Requests:** Single-user app; no concurrency testing needed
-

Limitations

Current Limitations

1. LLM Model Size

Issue: Small local models (qwen2.5:3b) occasionally select wrong columns or propose invalid joins

Evidence: ~10–15% of complex queries require user correction

Workaround:

- Always review generated SQL in expandable "SQL used" block
 - Use larger model (qwen2.5:7b) for complex queries (requires 8 GB+ RAM)
 - Rephrase question with explicit table/column names
-

2. Single-Language Voice Input

Issue: Whisper (tiny.en) is English-only; no multilingual support

Scope: Field terminology in other languages not recognized

Workaround:

- Type questions in local language
 - Use base.en model for better English accuracy
 - Translate field terms to English before recording
-

3. No Query History

Issue: Session resets; previous queries not saved across refreshes

Impact: Cannot review past analysis; no reproducibility across days

Workaround (Recommended Addition):

```
# Add persistent logging
import json
from datetime import datetime

def log_query(question, sql, row_count):
    log_entry = {
        "timestamp": datetime.now().isoformat(),
        "question": question,
        "sql": sql,
        "rows": row_count
    }
    with open("query_history.jsonl", "a") as f:
        f.write(json.dumps(log_entry) + "\n")
```

4. No User Authentication

Issue: App assumes trusted, single-user environment

Risk: Anyone with URL access can view all data; no audit trail of who ran what query

Recommended Production Addition: Streamlit authenticator (see Security section)

5. No Persistent Configuration

Issue: `.env` must be manually updated; no UI for changing models

Workaround: Edit `.env` and restart app manually

6. Limited Error Messages

Issue: If Ollama is down or model not found, error message is generic

Example: "Ollama returned an invalid response"

Improvement Needed: Detect specific errors (connection refused, model not found, etc.) and provide actionable messages

7. 200-Row Default Limit

Issue: Large result sets (>200 rows) are truncated; user cannot easily see all results
Workaround: Change MAX_ROWS in `sql_safety.py` or ask for filtered query

Known Issues

Issue	Frequency	Impact	Workaround
Ollama takes 30+ sec on first query	Always	Cold start latency	Warm up Ollama by running a simple query on app load
Whisper downloads on first use	Always	One-time 75 MB download	Pre-download model or disable voice if offline
Chat messages don't persist after refresh	Always	Session state lost	Implement database logging (future enhancement)
Non-English table names cause errors	Rare	Ollama cannot generate correct SQL	Use English table/column names

Future Roadmap

Short-Term (1–3 months)

1. Persistent Query History

```
# Store all queries in SQLite database
CREATE TABLE query_history (
  id INTEGER PRIMARY KEY,
  timestamp DATETIME,
  question TEXT,
  sql TEXT,
  row_count INTEGER,
  execution_time FLOAT
);

# Enable users to:
- View past queries
- Re-run saved queries
- Export query history as CSV
```

2. Query Bookmarking

```
# Save favorite queries for quick re-execution
```

- "Average RootLength by FruitShape" → click to re-run
- Bookmark sync across sessions

3. Multi-User Authentication

- ```
Integrate Streamlit authenticator
- Username/password login
- Query audit trail (who ran what, when)
- Role-based access (breeding program lead, technician, etc.)
```

---

## Medium-Term (3–6 months)

### 4. Breeding-Domain Data Dictionary

```
Add metadata for plant-specific terms
{
 "tables": {
 "Root_fst": {
 "description": "First root measurement stage (V6 growth stage)",
 "date": "2024-06-15",
 "sample_size": 150
 },
 "columns": {
 "R_S_Ratio": {
 "full_name": "Root-to-Shoot Ratio",
 "units": "dimensionless",
 "measurement_method": "Fresh weight"
 }
 }
 }
}
```

### 5. Advanced Visualization

- Export charts as publication-ready PDF
- 3D scatter plots for multi-variate analysis
- Interactive trait correlation matrix
- Time-series plotting (if measurement dates added)

### 6. Multi-Table Import Assistant

- ```
# UI wizard for bulk CSV import
- Select CSV files
- Auto-map columns to known fields
- Preview rows before import
```

```
- Create MySQL tables automatically
```

Long-Term (6–12 months)

7. Comparative Genomics Integration

```
# Link genotype performance to molecular markers
- Import SNP genotype data
- Correlate phenotype (root, physiology) with genotype
- Identify marker-trait associations
```

8. Statistical Analysis Module

```
# Automated statistical testing
- ANOVA: Compare traits across genotypes
- Heritability estimation
- QTL mapping (if genetic data available)
- Principal Component Analysis (PCA) for trait reduction
```

9. Custom Model Fine-Tuning

```
# Fine-tune Ollama on breeding domain
- Collect successful queries as training examples
- Fine-tune qwen2.5:3b on plant-breeding SQL
- Expected improvement: 95%+ SQL accuracy
```

10. Mobile App & Offline Mode

- React Native app for iOS/Android
- Offline query builder (generate SQL without Ollama)
- Sync results when online

Appendix

Example Queries Reference

Example 1: Simple Table Preview

Question: "Show the first 10 rows from Root_fst"

Generated SQL:
SELECT * FROM `Root\_fst` LIMIT 10;

Result: First 10 rows of Root_fst table

Example 2: Single-Table Aggregation

Question: "What is the average RootLength in Root_fst?"

Generated SQL:
SELECT AVG(RootLength) AS AverageRootLength
FROM Root_fst;

Result: Single metric card showing "AverageRootLength: 28.54"

Example 3: Grouped Aggregation

Question: "Show average RootLength grouped by FruitShape"

Generated SQL:
SELECT FruitShape, AVG(RootLength) AS AvgRootLength
FROM plant_morpho pm
INNER JOIN Root_fst rf ON pm.GenotypeName = rf.GenotypeName
GROUP BY pm.FruitShape
ORDER BY AvgRootLength DESC
LIMIT 200;

Result: Bar chart (FruitShape on X, AvgRootLength on Y) + table

Example 4: Multi-Table JOIN

Question: "Compare RootLength and SPAD for each GenotypeName, ordered by RootLength"

Generated SQL:
SELECT rf.GenotypeName, rf.RootLength, pf.SPAD
FROM Root_fst rf
INNER JOIN phy_fst pf ON rf.GenotypeName = pf.GenotypeName
ORDER BY rf.RootLength DESC

```
LIMIT 200;
```

```
Result: Table + optional scatter plot (RootLength vs SPAD)
```

Example 5: Frequency Distribution

```
Question: "Count plants by FruitColour"
```

```
Generated SQL:
SELECT FruitColour, COUNT(*) AS Count
FROM plant_morpho
GROUP BY FruitColour
ORDER BY Count DESC;
```

```
Result: Frequency bar chart + table showing color distribution
```

Example 6: Multi-Stage Comparison

```
Question: "Show RootLength changes from first to second measurement for each genotype"
```

```
Generated SQL:
SELECT
    rf.GenotypeName,
    rf.RootLength AS FirstRootLength,
    rs.RootLength AS SecondRootLength,
    (rs.RootLength - rf.RootLength) AS Growth
FROM Root_fst rf
INNER JOIN Root_scd rs ON rf.GenotypeName = rs.GenotypeName
ORDER BY Growth DESC
LIMIT 200;
```

```
Result: Table showing root development; scatter plot option (First vs Second)
```

Glossary

Term	Definition
GenotypeName	Unique identifier for each eggplant breeding line

Term	Definition
SPAD	Soil Plant Analysis Development value; proxy for leaf chlorophyll content
R_S_Ratio	Root-to-Shoot Ratio; measure of root vigor relative to above-ground biomass
MSI	Membrane Stability Index; measure of cell membrane integrity under stress
RWC	Relative Water Content; measure of plant water status
Canopy Temperature	Leaf/canopy temperature measured with thermal camera; indicates water stress
Epicuticular Wax	Waxy coating on leaves; affects drought tolerance
FruitShape	Categorical descriptor (e.g., Round, Oval, Long)
ART Framework	Action–Role–Task; structured prompt methodology for LLMs
Ollama	Open-source LLM inference engine; runs models locally
Whisper	OpenAI's speech-to-text model; runs locally via faster-whisper
sqlglot	SQL parsing and validation library
Streamlit	Python web app framework for data apps
SQLAlchemy	Python ORM for database access

Support & Contribution

Reporting Issues

If you encounter bugs or unexpected behavior:

1. Check the troubleshooting section in Installation Guide
2. Review generated SQL (expandable "SQL used" block)
3. Check MySQL is accessible; Ollama is running
4. Open an issue on GitHub with:
 - Question asked
 - Generated SQL (if shown)
 - Error message
 - Environment (OS, Python version, MySQL version)

Contributing Enhancements

Contributions are welcome. Please:

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/my-improvement`)

3. Write tests for new functionality
4. Submit a pull request with description

References & Resources

- **Ollama Documentation:** <https://ollama.com>
- **Streamlit Documentation:** <https://docs.streamlit.io>
- **SQLAlchemy ORM:** <https://docs.sqlalchemy.org>
- **sqlglot SQL Parsing:** <https://github.com/tobymao/sqlglot>
- **faster-whisper:** <https://github.com/guillaumekln/faster-whisper>
- **MySQL Documentation:** <https://dev.mysql.com/doc>

License & Attribution

Project License: [Insert your license here, e.g., MIT, Apache 2.0]

Image Attribution: Eggplant varieties photograph by J.E. Fee, via Wikimedia Commons, licensed CC BY 2.0.

Third-Party Libraries: See `requirements.txt` for all dependencies and their licenses.

Contact & Support

- **GitHub Repository:** https://github.com/TECHNEWBI5Z/Analytical_ChatBot
- **Issues & Feature Requests:** Open an issue on GitHub
- **Email Support:** [Insert contact email if applicable]

Document Version: 1.0

Last Updated: January 2025

Prepared For: Onboarding, Deployment, Maintenance

End of Documentation

This comprehensive documentation serves as the single source of truth for the **Brinjal Insights** project. It is intended for:

- **New Developers:** Understanding architecture, workflow, and codebase
- **DevOps/System Administrators:** Deployment, configuration, and maintenance
- **Stakeholders:** Features, use cases, and project roadmap
- **Plant Breeders:** How to ask questions and interpret results

For questions or updates, refer to the GitHub repository or contact the project maintainers.